

Prueba Corta #1 – Git y Organización Archivos

Nombre: Angela Jimena Santizo Escobar

Califica: _____ Punteo: _____

Instrucciones: Seleccionar la opción o las opciones correctas de cada enunciado.

1. **¿Qué se entiende por organización de archivos?**
 - a) La forma en que los registros se almacenan y se organizan dentro de un archivo
 - b) La forma en que se diseñan las interfaces de usuario
 - c) El proceso de compilar un programa
2. **¿Cuál es una característica de la organización de archivos tipo pila?**
 - a) Los registros siempre deben estar ordenados
 - b) Los registros se agregan generalmente al final del archivo
 - c) Cada registro necesita obligatoriamente un índice
3. **¿Qué caracteriza al acceso secuencial?**
 - a) Los registros se procesan uno después de otro
 - b) Permite acceder directamente a cualquier registro sin recorrer los anteriores
 - c) Los registros deben tener un índice para poder ser consultados
4. **¿Cuál es una ventaja del acceso directo?**
 - a) Requiere recorrer todos los registros anteriores
 - b) Solo puede utilizarse con archivos de texto
 - c) Permite localizar rápidamente un registro específico
5. **¿Qué función cumple un índice en una organización secuencial indexada?**
 - a) Convertir el archivo en una base de datos
 - b) Facilitar la localización de registros dentro del archivo
 - c) Eliminar automáticamente los registros que no se utilizan
6. **Una empresa tiene un archivo con miles de clientes y necesita consultar rápidamente la información de un cliente utilizando su número de cliente. ¿Qué opción sería más adecuada?**
 - a) Acceso secuencial obligatorio
 - b) Organización tipo pila sin búsqueda
 - c) Acceso directo
7. **Se modifican Program.cs y Usuario.cs. Se ejecuta: git add Program.cs / git commit -m "Corrige programa". ¿Qué ocurre?**
 - a) Se envían los archivos a GitHub
 - b) Se registra únicamente el cambio de Program.cs
 - c) Se registran ambos archivos
8. **git status muestra: modified: Login.cs / modified: Usuario.cs. Se quiere hacer commit solo de Login.cs. ¿Qué opción es correcta y por qué?**
 - a) git push Login.cs
 - b) git add . → git commit -m "Corrige login"
 - c) git add Login.cs → git commit -m "Corrige login"
9. **¿Qué hace este flujo, explique? git add . / git commit -m "Nueva funcionalidad" / git push**
 - git add . → agrega todos los archivos modificados/nuevos al área de staging.
 - git commit -m "Nueva funcionalidad" → crea un commit local con esos cambios y el mensaje que se le agrego.
 - git push → sube ese commit al repositorio remoto, actualizando la rama remota con los cambios locales.
10. **Se quiere trabajar una funcionalidad sin modificar main. ¿Qué comando crea la rama?**
 - git checkout -b nombre-rama. Esto crea una nueva rama a partir del estado actual sin tocar main, permitiendo trabajar en esta nueva rama sin modificar nada de la principal.

11. Después de: `git clone URL` / `git add .` / `git commit -m "Actualiza usuarios"`.

¿Qué falta para enviar el commit al remoto?

- Falta hacer `git push` para enviar el commit local al repositorio remoto. Ya que el `git add` y `git commit` solo registran los cambios en el repositorio local y con el `git push` ya se sube todo al repositorio remoto.

12. Otro desarrollador subió cambios al repositorio remoto.

¿Qué comando permite traerlos?

- Un `git pull` es muy importante al abrir un proyecto compartido porque con ese comando jalamos todos los cambios que hizo el otro colaborador en el remoto y los integra en la rama local. También se puede usar `git fetch` solo para descargar sin fusionar automáticamente.

13. Se tienen las ramas: `main: A---B---C` / `feature: A---B---D---E`. Se quiere integrar `feature` en `main`.

¿Qué concepto o comando se utiliza y por qué?

- Se usa `merge`, porque `main` y `feature` tienen en común (B) pero cada una avanzó por su lado (C en `main`, D-E en `feature`). El `merge` combina ambas ramas creando un commit de fusión que une los cambios.

14. ¿Cuál configuración establece correctamente la identidad de Git?

a) `git user --name "Ana" / git user --email "ana@email.com"`

b) `git config --global user.name "Ana" / git config --global user.email "ana@email.com"`

c) `git setup user.name "Ana" / git setup user.email ana@email.com`

15. ¿Qué comando permite revisar la rama actual y los cambios pendientes?

- `git status`. Muestra la rama actual, los archivos modificados, los que están en `staging` y los que no están siendo rastreados. Muestra un panorama de todo.

16. Un commit local no está en GitHub y la rama está adelantada respecto al remoto.

¿Qué comando se utiliza y por qué?

- `git push`. Se usa porque hay un commit local que el remoto no tiene entonces con el `push` se sube el commit que acabamos de hacer sincronizando el repo local con el remoto.

17. ¿Qué hacen estos comandos? `git branch feature-pagos` / `git checkout feature-pagos`

- `git branch feature-pagos` — crea una nueva rama llamada "feature-pagos" a partir del commit actual.
- `git checkout feature-pagos` — Se mueve a trabajar en esta rama

18. Hay cambios locales y se quiere revisar cuáles están modificados y cuáles preparados para commit. ¿Qué comando se utiliza?

- `git status`. Muestra qué archivos están modificados y cuáles ya están en el área de `staging` listos para commit.

19. Se quiere integrar `feature-login` en `main`. ¿Cuál opción es correcta? Explique con sus palabras.

a) Ejecutar `git delete feature-login`

b) Cambiar a `main` y ejecutar `git merge feature-login`

c) Ejecutar `git clone feature-login`

20. Estando en `main`, se quiere integrar la rama `feature`: `main: A---B---C` / `feature: A---B---D`. ¿Qué comando se utiliza y por qué?

- Se usa `git merge feature`. En este caso, como `main` (A-B-C) y `feature` (A-B-D) unieron después de B, Git necesita crear un commit de fusión que combine C y D, conservando el historial de ambas ramas.